

Document Review

Editorial & technical review of the **AWS Cost Detective** project overview

DOCUMENT REVIEWED	AUTHOR	REVIEW DATE	PAGES
AWS_Cost_Detective_Overview.pdf	Suchith Jain	August 4, 2026	3

Overall assessment

Strong

A clean, credible, well-designed one-pager. The architecture is easy to grasp and the "AI only narrates, rules decide" principle is exactly the right thing to lead with. **Two true contradictions** and a handful of **clarity gaps** should be resolved before sharing widely — none are structural; each is a small, quick edit.

What's strong

- KEEP

The "assembly line" diagram
The SOURCE → Collector → Analyzer → Reasoner → FastAPI → You flow makes the whole architecture graspable in about five seconds. Best element in the document.
- KEEP

The "Why the AI is boxed into one step" callout
Genuinely excellent. Stating that detection is deterministic (rules in `analyzer.py`) and the LLM *only narrates* a confirmed finding preempts the #1 skeptical question about any "AI + money" tool: *can it hallucinate a fake cost?* The answer — no — is your strongest selling point. Keep it prominent.
- KEEP

Consistent read-only / mock-mode framing
Repeated across the callout, prerequisites, and run table. It lowers the barrier to trying the tool and addresses the safety concern up front.
- KEEP

Professional, concrete tone
Real file names (`collector.py` , `models.py` , `pricing.py`) give it credibility over hand-wavy marketing language.

Gaps & inconsistencies to fix

Each point below explains the underlying concept in plain terms, why it matters, and the concrete fix.

CONTRADICTION 1 · The frontend stack is described two different ways

The **frontend** is the part users see and click in the browser. The Tech-stack section calls it "**React + Vite (NEXT)**", but the Live-app section calls it a single `frontend/index.html` file. These are fundamentally different: a plain `index.html` just opens in a browser with no build step, whereas React / Vite / Next.js are three separate, heavier tools that must be installed and built first. A reader can't tell whether this is a simple one-file page or a full app that needs a build — and those have completely different setup steps.

Fix: Pick the one that's true and state only that. If it's really `index.html`, remove "React + Vite (NEXT)."

CLARITY 2 · The `/scan` endpoint is shown inconsistently

An **endpoint** is the specific web address the frontend calls to make the backend run — here, `/scan`. Some sections show `/scan?region=us-east-1`. The `?region=` tail isn't a different endpoint; it's just an option telling `/scan` which region to inspect. Shown inconsistently, a beginner may think there are several different endpoints.

Fix: Use `/scan` everywhere, and explain the `?region=` option once.

CONTRADICTION 3 · "OpenAI is optional" is stated two conflicting ways

An **API key** is a password that lets the tool talk to an outside AI service, used here for one small job: rewriting a finding into friendly English. The prerequisites correctly call the key **optional** ("works fully without one"), but the Tech-stack tile lists "**OpenAI API**" as a core piece with no "optional" label. A beginner reading the tile may wrongly think they must sign up for (and pay) OpenAI before anything runs.

Fix: Add "(optional)" to the AI tile so both statements agree. Note: since the job is just rewriting text, any capable LLM (e.g. Claude) fits — consider provider-neutral wording.

CLARITY 4 · "Rough" prices vs. an exact-looking dollar figure

`pricing.py` is honestly called a "**rough**" price table, yet the headline shows a precise-looking "**\$207.15 recoverable every month.**" Showing cents implies exactness, and AWS prices drift over time, so a hardcoded table can quietly go stale. If a user acts on "\$207.15" and really saves \$150, trust in the whole tool drops.

Fix: Present it as an estimate — e.g. `~$207/month (estimated)` — and add one line on how/when the price table is refreshed.

CLARITY 5 · Only four detectors — say whether that's the whole story

The tool detects four things (over-provisioned servers, unused disks, gp2 → gp3, idle IPs), presented as the complete list. But the architecture makes adding checks easy — each is just a rule in `analyzer.py`. As written, "only 4 checks" reads as a limitation rather than a foundation.

Fix: Add a sentence: "New checks are simple rules added to `analyzer.py`, so the detector list is easy to extend."

CLARITY 6 · Missing practical basics a first-time user needs

Two questions the document never answers: **(a) Which region(s) does it scan?** AWS splits the world into regions (e.g. `us-east-1`); if the tool checks one at a time, a user's resources may live in a region it never looks at, so it reports "\$0" and seems broken. **(b) Does scanning cost anything or hit limits?** AWS caps how many questions you can ask per second (a "rate limit"); one reassuring sentence would help on large accounts.

Fix: Add: "Scans one region at a time (set with `?region=`). Reading resource info is free and stays within AWS's normal limits."

Minor copy nits

- **"for no good reason"** (What it does) reads slightly casual against the otherwise professional tone. Optional.
- The screenshot caption says the demo "always looks good" because mock data appears when the backend is unreachable — a nice UX touch, but it could read as "hides real failures." Consider **"so the demo works offline"** instead.

Priority summary

PRIORITY	ITEM	TYPE
Fix first	#1 Frontend stack contradiction	Contradiction — a technical reader catches it immediately
Fix first	#3 OpenAI "optional" inconsistency	Contradiction — conflicting claims
Soon	#4 Estimate vs. exact price	Clarity / trust
Soon	#6 Region & scan-limit basics	Clarity / usability
Polish	#2 Endpoint naming, #5 Extensibility note, copy nits	Nice-to-have

Bottom line

A solid, credible overview. The architecture and the "AI narrates, rules decide" principle are the real differentiators — and you already lead with them. Resolve the two contradictions (#1 frontend stack, #3 OpenAI optional) before sharing widely, since those are what a technical reader will spot first. Everything else is a one-sentence addition that turns a good document into a polished one.